



INSTITUTO POLITÉCNICO
UNIVERSIDADE KIMPA VITA
DEPARTAMENTO ENGENHARIA INFORMÁTICA

TRABALHO ACADÉMICO DE SGBD II

Conceção e Implementação de uma Camada de Persistência Poliglota para um Ecossistema de E- commerce de Alta Escala

Motor de Recomendações e Redes Sociais NoSQL

Membros do Grupo (3º Ano):

1. Alegria Neves Caluvi
2. Cláudia Armando
3. Garcia Lopes Camalando
4. Rosário Canga
5. Nelson Manuel Pedro Teca

Docente: Prof. Moyo Kanivengidio

Uíge, 2026

Índice

1. Introdução e Enquadramento Tecnológico

2. Análise de Requisitos e Seleção Tecnológica

3. Arquitetura e Modelação de Dados no Neo4j

4. Estratégia de Povoamento de Dados e Cenário Simulado

5. Algoritmos de Filtragem e Sistema de Recomendações

6. Análise Crítica: Desempenho, Escalabilidade e Teorema CAP

7. Conclusão e Resultados Obtidos

8. Referências Bibliográficas e Entregáveis

1. Introdução e Enquadramento Tecnológico

No atual panorama tecnológico, caracterizado pelo advento do *Big Data*, aplicações distribuídas altamente resilientes e a ubiquidade dos microsserviços, as organizações são frequentemente confrontadas com o desafio de processar, armazenar e extrair valor de volumes imensos de dados em tempo real. Os Sistemas de Gestão de Bases de Dados Relacionais (RDBMS) tradicionais, embora tenham sido o padrão de ouro durante décadas para garantir propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade), apresentam severas limitações arquiteturais quando se trata de escalabilidade horizontal e flexibilidade de esquemas para dados polimórficos.

Para colmatar estas falhas estruturais nos cenários contemporâneos, emergiram as abordagens de persistência **NoSQL** (*Not Only SQL*). Estas tecnologias abandonam a rigidez das tabelas normalizadas e abraçam diferentes paradigmas de modelação, tais como os armazenamentos do tipo Chave-Valor, Bases de Dados Orientadas a Documentos, Famílias de Colunas e, fundamentalmente para o escopo deste trabalho, as Bases de Dados Orientadas a Grafos.

Este relatório técnico formal, desenvolvido no rigoroso contexto académico da disciplina de **Sistemas de Gestão de Bases de Dados II (SGBD II)**, expõe detalhadamente o ciclo completo de conceção e implementação de uma camada de

persistência avançada para um subdomínio nevrálgico num ecossistema de E-commerce de alta escala: o **Motor de Recomendações e Redes Sociais**.

Numa plataforma de comércio eletrônico competitiva contemporânea, garantir uma jornada de utilizador fluida, altamente cativante e hiper-personalizada já não é um luxo, mas sim um requisito de sobrevivência do negócio. O motor de recomendações ideal deve ser capaz de cruzar, numa questão de milissegundos, o vasto histórico de transações passadas com as interações sociais e preferências latentes dos utilizadores. É neste desafio de correlação de múltiplas entidades complexas que a escolha de uma base de dados orientada a grafos justifica o seu valor, anulando a pesada penalização computacional dos tradicionais *JOINS* relacionais.

Adotando uma perspetiva holística de *Project-Based Learning*, conforme as rigorosas diretrizes pedagógicas estabelecidas pelo docente orientador **Prof. Moyo Kanivengidio**, o grupo assumiu o papel proativo de Arquitetos de Dados. As secções seguintes pormenorizam os fundamentos da escolha tecnológica, a modelação (Query-Driven

Design), o rigoroso processo de povoamento massivo simulado em, os resultados práticos da implementação de pesquisas Cypher, e a respetiva análise crítica de desempenho ^{Python} alicerçada nas leis dos Teoremas CAP e PACELC.

2. Análise de Requisitos e Seleção Tecnológica

2.1. O Desafio Crítico das Recomendações no E-commerce

Ao destringir o domínio do "Motor de Recomendações e Redes Sociais" integrado num ecossistema comercial global, identificamos requisitos não-funcionais e funcionais de altíssima exigência:

- **Profundidade Relacional e Consultas Multi-Hop:** O sistema tem de conseguir inferir padrões recursivos, por exemplo: *"Descobrir todos os produtos da Categoria 'Eletronicos' que foram comprados por utilizadores que, por sua vez, são amigos de pessoas que compraram o 'Produto X' nas últimas 48 horas"*.
- **Desempenho em Tempo Real (Filtragem Colaborativa):** A latência de consulta não pode degradar à medida que o tamanho da base de dados cresce. Identificar

utilizadores com afinidades e gostos similares deve ocorrer numa janela de tempo inferior a 50 milissegundos.

- **Write-Intensive vs. Read-Intensive:** Embora um e-commerce sofra milhares de writes por segundo (compras, novas amizades), o subsistema de recomendações será primordialmente *Read-Heavy* (frequentemente acedido sempre que um cliente carrega a página inicial ou visualiza o detalhe de um produto).

2.2. O Paradigma Relacional vs. O Paradigma NoSQL

Num SGBD Relacional (como PostgreSQL ou MySQL), mapear as interações sociais e compras requer tabelas associativas e a execução de `JOINS` dispendiosos. Quando a rede de conexões aumenta (escalabilidade vertical e horizontal do grafo), as tabelas de associação tornam-se gigantescas. Calcular o grau de separação entre dois utilizadores torna-se uma operação de força bruta altamente bloqueante para o servidor.

Por outro lado, avaliar ferramentas NoSQL convencionais como o **MongoDB** (Base de Dados de Documentos) revelou também as suas próprias fraquezas neste contexto. O MongoDB brilha em agregação de volumes de dados hierárquicos e autocontidos (ex: um catálogo de produtos detalhado num único ficheiro JSON). Contudo, a modelação de cardinalidades densas (muitos-para-muitos) obriga o programador a armazenar *Arrays de Referências*, o que muitas vezes exige ao *middleware* (a aplicação em Python ou Node.js) realizar múltiplas chamadas consecutivas para a base de dados para "montar" a recomendação, um anti-padrão conhecido como *N+1 query problem*.

2.3. A Seleção Fundamentada: O Triunfo do Neo4j

Tendo em conta as limitações expostas, a equipa selecionou categoricamente o **Neo4j**, líder incontestável no mercado de bases de dados de grafos orientadas por propriedades (*Property Graphs*). A sua adoção assenta em três pilares fundamentais:

1. **Index-Free Adjacency (Adjacência Livre de Índices):** Ao contrário dos SGBDs Relacionais que utilizam Índices B-Tree para encontrar as linhas correspondentes num JOIN (custo $O(\log N)$), o Neo4j gere fisicamente as ligações entre nós em memória, através de ponteiros diretos (custo $O(1)$ por salto). O tempo de travessia do grafo depende apenas da quantidade de nós percorridos na vizinhança local da pesquisa, ignorando o volume macroscópico dos dados armazenados.
2. **Linguagem Cypher:** Uma linguagem declarativa e expressiva desenhada expressamente para expressar padrões complexos de conexões em formato de ASCII-Art visual (ex: `(NóA) - [Ligação] -> (NóB)`).
3. **Algoritmos Nativos de Grafos:** O ecossistema fornece acesso rápido (via *Graph Data Science Library*) a funções como PageRank, deteção de comunidades e similaridade de

cossenos, que poderiam enriquecer largamente futuras versões do projeto.

3. Arquitetura e Modelação de Dados no Neo4j

A metodologia adotada para desenhar as estruturas em sistemas NoSQL inverte a lógica relacional clássica. Em vez de modelar em torno da eliminação de redundâncias (Formas Normais de Codd), adotou-se a filosofia de **Query-Driven Design**: o modelo de dados constrói-se tendo em mente as exatas perguntas que necessitamos de colocar ao SGBD para alimentar o painel do utilizador.

3.1. Desenho do Esquema (Property Graph Model)

A nossa ontologia computacional para o domínio foi sintetizada em duas *Labels* primárias (Entidades) que encapsulam as propriedades fundamentais do contexto e-commerce:

- **Nó `User` (O Utilizador/Cliente)**: Representa o ator dinâmico da plataforma. As suas propriedades incluem um identificador global (`userId`), o seu nome formatado e a respetiva `cidade` .
- **Nó `Product` Mercadoria**: A representação tangível do inventário. Possui o `productId` o nome descritivo (ex: "Teclado Mecânico") e uma categorização ampla (ex: "Eletronicos").

3.2. Mapeamento Semântico e Relacionamentos (Edges)

No universo dos grafos, as arestas (relacionamentos) assumem um estatuto de primeira classe, possuindo tanto significado quanto os próprios nós que ligam.

- `(User) -[:COMPROU]->(Product)` : Uma ligação direta e unidirecional. Fundamental para a extração do histórico e construção de recomendações do tipo item-para-item. Pode no futuro armazenar propriedades como a data de compra e o rating (estrelas) deixado ao item.
- `(User) -[:AMIGO_DE]->(User)` : O cimento social da plataforma. Embora no motor de geração tenha sido inserido de forma aleatória unidirecional, reflete a partilha de interesses e amizades. Ao navegar nesta relação, é possível recomendar algo baseado no filtro: *"8 dos teus amigos já compraram este produto, devias experimentar"*.

Ausência de Esquema Rígido (Schema-Optional): *A plasticidade do modelo NoSQL garantiu que a adição de novos atributos, como a localização, fosse impercetível à camada de aplicação, dispensando a complexa gestão de scripts de migração estrutural que paralisa sistemas tradicionais durante longos períodos de manutenção noturna.*

4. Estratégia de Povoamento de Dados e Cenário Simulado

De modo a criar um relatório válido para um contexto de "Alta Escala", a equipa rejeitou categoricamente a inserção trivial de apenas um punhado de nós ilustrativos. Tornou-se imperativo construir uma massa de dados crítica que espelhasse o ruído transacional de uma loja ativa.

4.1. O Gestor de Conexões em Python (conexão.py)

Para garantir o isolamento e reutilização da comunicação com o servidor Neo4j via protocolo binário *Bolt* (correndo na porta padrão 7687), desenvolvemos o módulo central de abstração de driver:

```
# Ficheiro: conexão.py
from neo4j import GraphDatabase

URI = "bolt://localhost:7687"
USERNAME = "neo4j"
PASSWORD = "valdemiro" # Credenciais encriptadas no ambiente real

driver = GraphDatabase.driver(
    URI,
    auth=(USERNAME, PASSWORD)
)

def close_connection():
    driver.close()
```

4.2. Geração Randômica Volumétrica (geradorDados.py)

O script `geradorDados.py` recorre à biblioteca **Faker** (um standard de indústria em Test Driven Development para injeção de dados falsos que pareçam realistas). A execução dividiu-se em transações isoladas para optimização da cache:

1. **Injeção de Clientes** `create_usuarios()` : Iterando de 0 a 10.000, gerando um dicionário vibrante e localizado de nomes e cidades, mapeados diretamente para a instrução Cypher `CREATE (:User {{userId: $id, nome: $name...}})`
2. **Injeção Transacional** `create_compras()` : Simulação agressiva de compras. O algoritmo seleciona aleatoriamente IDs do universo dos 10.000 utilizadores e os vincula a 20 produtos chave pré-definidos (abrangendo "Roupas", "Esportes", "Eletronicos", e "Calçados"). Ao gerar cerca de **50.000 eventos de compra**, garantimos a presença de *power-buyers* e de produtos *best-sellers*.
3. **Injeção de Grafo Social** `create_amizades()` : Construção de uma teia relacional complexa, ligando 70.000 pares aleatórios de utilizadores através do verbo

`[ :AMIGO_DE ]`. O facto de garantir a verificação de entropia (evitando que um nó seja amigo dele mesmo) garante a solidez semântica do modelo.

Após a execução desta rotina, o servidor de base de dados atingiu uma profundidade que totalizou **mais de 120.000 nós e ligações interligadas**.

5. Algoritmos de Filtragem e Sistema de Recomendações

A joia da coroa deste projeto reside no ficheiro `sistRecomendacoes.py`, que alberga a inteligência da filtragem colaborativa construída com as fundações da teoria dos grafos.

5.1. Filtragem Colaborativa Baseada em Itens

Este método inferencial não exige compreender *o que é* o produto fisicamente, mas assume que *se o grupo de utilizadores Y consome a combinação A+B, e o utilizador X consome A, há forte probabilidade de X também apreciar B*. Este é um mecanismo *peer-to-peer* extremamente popularizado pela Amazon e pelo algoritmo de sugestão da Netflix.

5.2. Dissecando a Consulta Cypher de Recomendação

A pesquisa desenhada atravessa duas dimensões em profundidade a partir do produto visado. Abaixo apresenta-se o bloco analítico de recomendação principal que a equipa arquitetou:

```

MATCH (u:User)-[:COMPROU]->(p1:Product)
WHERE p1.nome = $product

// O utilizador 'u' (que comprou o nosso produto base)
// comprou também um produto recomendado alternativo
MATCH (u)-[:COMPROU]->(recommended:Product)
WHERE recommended.nome <> $product

// Agregação estatística, ordenando pelo maior número de co-ocorrências
RETURN recommended.nome AS product,
       COUNT(*) AS score
ORDER BY score DESC
LIMIT 5

```

A mecânica algorítmica deste excerto:

- O motor pesquisa o catálogo e ancora-se no produto introduzido pelo utilizador final no prompt (e.g., "Placa de Vídeo").
- A partir deste nó raiz, a função percorre inversamente a seta `[:COMPROU]` para identificar a lista de todos os indivíduos (`u`) que adquiriram a Placa de Vídeo.
- Posteriormente, espalha-se novamente para a frente, observando todos os *outros* produtos que estes exatos indivíduos também compraram no passado.
- A cláusula `WHERE recommended.nome <> $product` garante a sanidade da resposta (evita recomendar ao utilizador uma Placa de Vídeo se ele já acabou de a comprar).
- O `COUNT(*)` age como uma agregação MapReduce transparente e nativa, criando um índice de popularidade de co-compra. A função 5" é `ORDER BY` garante que só o "Top fornecido à interface.

5.3. Interação do Utilizador em CLI

O script integra não só a arquitetura de backend como o tratamento exaustivo da Interface na Consola, listando os catálogos dinamicamente e bloqueando inputs mal formatados (ex: strings em vez de inteiros), protegendo assim as sessões do *driver* Neo4j e prevenindo potenciais ataques e falhas de ligação (via `try-except` abrangente).

6. Análise Crítica: Desempenho, Escalabilidade e Teorema CAP

6.1. Sucesso na Execução e Comparativo de Latência

O *output* console provou de forma pragmática a estabilidade do sistema ao imprimir imediatamente:

```
CONECTADO AO NEO4J COM SUCESSO. Total de nós no banco: [Valor] .
```

A percepção de resposta para obter as 5 recomendações críticas, após as seleções no catálogo gerado, deu-se num espaço temporal infra-segundo, revelando um dos grandes méritos do Neo4j.

Se tentássemos migrar a exata mesma lógica recursiva acima detalhada para tabelas SQL, a base de dados seria obrigada a executar varrimentos de tabela completos em tabelas associativas colossais, agrupamentos complexos em memória e junções cartesianas perigosas. A modelação em grafo revelou-se não só computacionalmente mais rápida, como cognitivamente e visualmente muito mais próxima da formulação natural da linguagem humana.

6.2. Enquadramento Conceptual nos Teoremas CAP e PACELC

A avaliação madura de um sistema distribuído requer posicioná-lo no prisma da restrição teórica de *Brewer*, vulgarmente conhecida como **Teorema CAP**: Num sistema distribuído sobre uma rede partilhável, é impossível garantir em simultâneo a Consistência absoluta (C), a Disponibilidade total (A) e a Tolerância total a Partições (P).

Num *deploy* distribuído do Neo4j (via Causal Clustering), o sistema alinha-se historicamente no espectro de **CP (Consistency & Partition Tolerance)**. Ao contrário das soluções baseadas em Documentos que favorecem Availability (AP), o Neo4j protege as transações (com ACID properties nativas no core). Para uma loja virtual, evitar que o sistema sugira produtos desfasados (inconsistentes com o stock temporal) é crítico. As garantias transacionais protegem as associações de grafo e evitam registos órfãos (links que não apontam para lado nenhum).

À luz da expansão teórica dada pelo **Teorema PACELC**, que responde à pergunta "*E quando não há partições de rede, o sistema favorece a Latência ou a Consistência (E, L or C)?*", o arquiteto pode ajustar as *read replicas* do Neo4j para aceitarem dados levemente desatualizados de forma a fornecer as recomendações o mais rápido possível ao cliente web (favorecendo a Latência sobre a Consistência Eventual).

6.3. Limitações Críticas da Solução

Com uma postura acadêmico-científica, o grupo destaca constrangimentos e oportunidades de evolução do protótipo:

- **Vácuo Analítico de Peso:** Todas as interações contam por 1 unidade estatística. Uma compra impulsiva e pontual é contabilizada com a mesma gravidade que uma interação repetida, ou uma classificação de "5 estrelas" a um produto. A introdução de `weight properties` nas setas corrigiria isso.
- **Falta de Contexto Temporal e Envelhecimento:** Uma recomendação baseada em compras feitas há 10 anos pode não ser pertinente no panorama atual de 2026. A integração de um *Decay Factor* na query cypher, baseado numa propriedade na compra, faria esquecer as conexões mais velhas em favor das mais recentes `timestamp`.
- **Filtros Baseados em Contexto de Utilizador Individual:** O motor de recomendação não tem em conta fatores demográficos da `cidade` gerada, perdendo assim uma oportunidade para recomendar produtos geograficamente mais aderentes.

7. Conclusão e Resultados Obtidos

O rigoroso trabalho de investigação e componente laboratorial levado a cabo na disciplina de SGBD II permitiu validar, de forma taxativa e prática, a importância do pensamento *Não Só SQL* na conceção de persistência para plataformas modernas de E-commerce. O grupo de trabalho, através da simulação num ambiente local via Python e Neo4j Driver, provou que a modelação baseada em redes, vértices e arestas é imbatível para resolver problemas correlacionais profundos, como é o caso explícito das Recomendações ao Consumidor e das Afinidades Sociais.

O script de inserção gerou em instantes milhares de ligações sintéticas aleatórias de clientes, reproduzindo um cenário denso de *Big Data*. As consultas Cypher desenvolvidas, apesar da sua aparente simplicidade sintática visual, comportam um motor de pesquisa relacional poderoso que entrega algoritmos de Filtragem Colaborativa quase instintivamente.

Em última análise, os Arquitetos de Dados desta equipa concluem que a Persistência Poliglota é o futuro irrefutável do desenvolvimento aplicacional — onde o Neo4j opera não para substituir as velhas tabelas relacionais em processos contabilísticos e fiscais, mas sim como uma camada suplementar analítica e rápida, dedicada puramente a entregar valor onde a estrutura importa mais do que o dado isolado.

8. Referências Bibliográficas e Entregáveis

8.1. Estrutura e Roteiro do Repositório (Anexos Técnicos)

De forma a cumprir integralmente as alíneas propostas na pauta avaliativa, as componentes de entrega formal do projeto assentam no seguinte repositório e infraestrutura de ficheiros:

- **Relatório_Documental.pdf**: O presente manifesto técnico, redigido no formato formal, detalhando justificações metodológicas, modelos e conclusões.
- **geradorDados.py**: Script em Python que utiliza a biblioteca Faker para efetuar a transação e simulação maciça do preenchimento da base de dados do E-commerce.
- **sistRecomendacoes.py**: Solução interativa por terminal que detém a formulação lógica de recomendações *cross-item*.
- **conexão.py**: Conector estrutural simplificado e parametrizado.
- **README.md**: Guia mestre (Playbook) desenhado para o reprodutibilidade do projeto. Detalha comandos de ambiente, pacotes Python (`pip install neo4j faker`)

para isolamento e levantamento suave do SGBD na máquina de teste do Docente.

8.2. Literatura Técnica

- Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph Databases: New Opportunities for Connected Data*. 2nd Edition. O'Reilly Media.
- Sadalage, P. J., & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional.
- Kanivengidio, M. (2025/2026). *Trabalho Prático: Arquitetura e Modelação Avançada em Sistemas de Gestão de Bases de Dados NoSQL*. Instituto Politécnico da Universidade Kimpa Vita. Uíge, Angola.
- Manual de Referência Cypher. Documentação Oficial do Neo4j (2026). Disponível em:
<https://neo4j.com/docs/cypher-manual/current/>